

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 1

UNSUPERVISED MACHINE LEARNING: CLUSTERING

Reading Time	65 minutes
Prior Knowledge	Linear Algebra, machine learning
Keywords	Clustering, K-means, elbow plot

In this lesson, we will learn the difference between supervised learning and unsupervised learning and their applications. Once we have a good understanding of unsupervised learning applications, we will study its foundational techniques. At the end of the lesson, we will implement the k-means clustering algorithm using built-in Python packages.

1. Introduction

In supervised learning, the output (target variable) that we want to predict is known. That allows us to assess the quality of our model by applying metrics such as recall, accuracy, mean squared error, etc., which are used to evaluate the performance of the model. In unsupervised learning, the target variable is not available. The unsupervised model helps in finding patterns in our clustered data without any associated response.

The table below lists some of the differences between unsupervised and supervised models.

Tables:

Unsupervised	Supervised
No labels provided	Labels are provided
Find or explore patterns in unlabeled data	Find or explore patterns in existing structure
Uses techniques such as clustering and dimensionality reduction	Uses techniques such as regression and classification

A supervised classification algorithm can be used to categorize given data into two known groups while an unsupervised clustering algorithm will tell us how the data is structured.

Unsupervised machine learning is applied under the following scenarios

- When the data has no output/target variable.
- When we want to understand patterns in the data.
- To pick out essential information (a process called dimensionality reduction) from the data and use it to train a supervised model.

There are a variety of applications of unsupervised machine learning in the financial industry. Here, we list just a few:

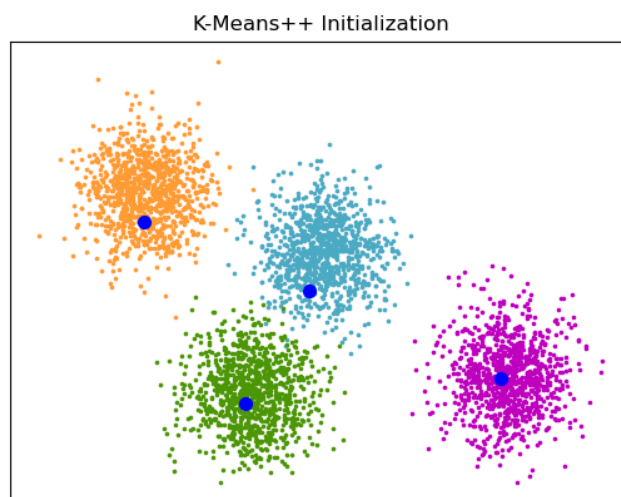
- Asset allocation
- Yield curve construction and interest rate modeling
- Trading to enhance speed and accuracy
- Pairs trading
- Portfolio management to cluster investors

In the rest of this lesson, we will study a branch of unsupervised machine learning called **clustering**.

2. Clustering

Clustering is a process of organizing data points into groups such that members of each group share some similar attributes. In the plot below, we can say that elements in Cluster 0 are similar to each other but different from elements in another cluster, say Cluster 1.

Fig 1: A Plot with Four Clusters



Source: [scikit-learn](#)

As seen in the above plot, we can cluster (group) the data points by using the distance metrics between adjacent datasets.

2.1 Distance-Based Clustering

Given a set of data points, we can use the distance between the points to group the points into clusters such that

- Internal distances between the individual points (intra-cluster) should be small, that is, the points of a given cluster should be close to each other.
- External distances between the points (inter-cluster) should be large, that is, the points in different clusters are dissimilar.

2.2 The Goals of Clustering

Given a set of unlabeled data, we will apply clustering to find the structure and groupings that exist in the data. There exists no perfect criterion for doing clustering, and it is upon the user to supply a criterion that will ensure the result matches their needs.

For any clustering method we opt to use, we need to define an appropriate **similarity measure** to be applied in finding the clusters.

2.3 Proximity Measures

When we do clustering, a proximity measure between two points must be defined where proximity is the data similarity or dissimilarity with respect to each other. The similarity measure $S(x_i, x_k)$ is large if the points x_i, x_k are similar while the dissimilarity measure $D(x_i, x_k)$ is small if the points x_i, x_k are similar.

Below are the similarity measures that we can use:

1. Vectors: Cosine Distance
$$s(\vec{x}, x') = \frac{x^t x'}{\|x\| \|x'\|}$$
2. Sets: Jaccard Distance
$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$
 If sets A and B are both empty, then we define $J(A, B) = 1$ otherwise $0 \leq J(A, B) \leq 1$
3. Points: Euclidean Distance
$$d(x, x') = \left(\sum_{k=1}^p |x_k - x'_k|^q \right)^{1/q}$$

2.4 Clustering Algorithms

Clustering algorithms can be categorized into four types as listed below:

1. Exclusive Clustering
2. Overlapping Clustering
3. Hierarchical Clustering
4. Probabilistic Clustering

In exclusive clustering, data is clustered exclusively such that a point belonging to one cluster can not be included in another cluster.

Overlapping clustering uses fuzzy sets to cluster data such that a point can belong to more than one cluster but with varying levels of membership. We therefore group data according to an appropriate membership value.

In hierarchical clustering, every data point is treated as a cluster and at each iteration we find the union between the two nearest clusters until we reach the number of clusters we wanted.

Finally, a probabilistic approach clusters each point based on their probability of belonging to a particular cluster.

Now let's look at some of the most commonly used clustering algorithms:

- K-means
- Fuzzy K-means
- Hierarchical clustering
- Mixture of Gaussians

K-means is an example of an exclusive clustering algorithm. Fuzzy K-means is an example of an overlapping clustering algorithm. Hierarchical clustering is just that, an example of a Hierarchical clustering algorithm. A mixture of Gaussians is an example of a probabilistic clustering algorithm.

Let's get a better understanding of each clustering method above.

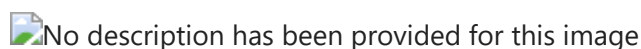
3. K-Means Clustering

K-means clustering works by classifying a dataset through a defined number of clusters that is fixed a priori.

Here is a walkthrough of the k-means algorithm:

1. We start by selecting k centroids where k is the number of clusters.
2. We then place the k centroids in our training data at different random places.
3. We then calculate the distance from each point in the training data to each centroid.
4. The points are then grouped based on the nearest centroid.
5. The grouped points are isolated together with their respective centroid. Find the mean of each group, then move the centroid to the location of the mean we have calculated.
6. We repeat the steps until we reach an optimal number of k groups.

Fig 2: A K-means clustering procedure flow chart



Source: Bustamam, A. et al. "Application of K-means Clustering Algorithm in Grouping the DNA Sequences of Hepatitis B Virus (HBV)." *AIP Conference Proceedings*, vol. 1862. no. 1. AIP Publishing LLC, 2017.

Mathematically, the objective of the K-means algorithm is to minimize an objective function which in our case is a squared error function given by $J = \sum_{j=1}^k \sum_{i=1}^n ||x_i^{(j)} - C_j||^2$ where

$\|x_i - C_j\|^2$ is the Euclidean distance between data points x_i and the centroid C_j .

Below is the mathematical walkthrough of the algorithm:

1. Let $X = \{x_1, x_2, \dots, x_n\}$ be our data points and $V = \{v_1, v_2, \dots, v_c\}$ be the centroids.
2. Select centroids randomly.
3. Then, calculate the distance between the centroids and each data point.
4. We then assign the data points to the centroids closest to them.
5. We recalculate the new centroids by computing the mean of the grouping, $v_i = \left(\frac{1}{c_i}\right) \sum_{j=1}^{c_i} x_j$ where c_i is the number of data points in the i th cluster.
6. Find the distance between the new centroid and each data point.
7. If there is no change in the composition of the clusters then we stop, or else we repeat the procedure from step 3.

The issue with the K-means algorithm is that the number of clusters need to be selected before we begin modeling. We can guess the number of clusters using either the Silhouette score or the elbow method.

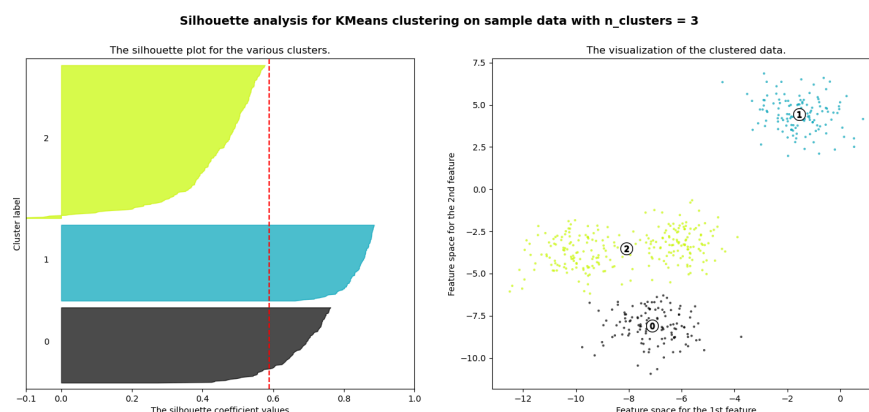
3.1 Silhouette Analysis

Silhouette analysis is applied in clustering to investigate the separation distance between our clusters. We use the Silhouette plot to visualize the distance of the points in one cluster to points in the neighboring clusters. The measure has a range of $[-1, 1]$.

Silhouette coefficients nearer to $+1$ show that the data points are far away from the adjacent clusters. A coefficient of 0 shows that the data points are very close to the neighboring clusters. Negative coefficients shows that the data points are probably assigned to the wrong cluster.

The Silhouette coefficient is given by $S = \frac{b-a}{\max(a,b)}$. An example of a Silhouette diagram is shown below.

Fig 3: Silhouette Analysis on Dataset with 3 Clusters



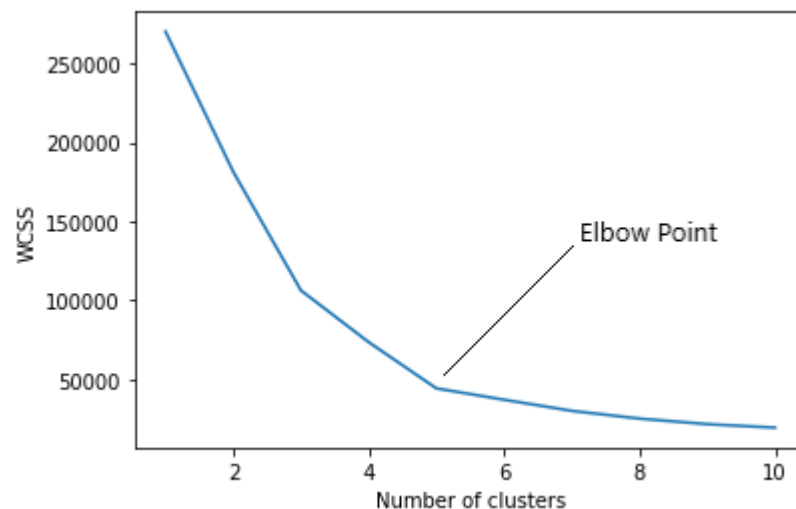
Source: [scikit-learn, Selecting the number of clusters with silhouette analysis on KMeans clustering](#)

3.2 Elbow Method

We can construct an elbow plot to find the appropriate value of k to use in our clustering.

We find the optimal number of clusters (groups) by drawing a line plot of Within the Sum of Squares (WSS) versus the number of clusters. We define WSS as the sum of square difference between each point in a cluster and its cluster center. An elbow plot is shown in the figure below:

Fig 4: An Elbow Plot Showing the Number of Optimal Clusters



Source: [Tripathi, Shreya et al. "Approaches to Clustering in Customer Segmentation." International Journal of Engineering & Technology, 2012, vol. 7, no. 3.12, 2018, pp. 802-807.](#)

Note that in the K-means algorithm, we do not have a general method for getting a unique optimal number of clusters.

We now put K-means into action with a simple example.

```
In [ ]: # Import packages

import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

np.random.seed(0)

%matplotlib inline
```

We generate a dataset with 4 clusters from the scikit-learn library 'make_blobs' and plot it for visualization.

```
In [ ]: X, y = make_blobs(centers=4, n_samples=2500)
```

```
fig = plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Dataset with 4 clusters", fontsize=20)
plt.xlabel("First feature")
plt.ylabel("Second feature")
plt.suptitle(
    "Fig. 5: Dataset Visualization", fontweight="bold", horizontalalignment="right"
)
plt.show()
```

In the plot above, we see that some clusters intersect with each other and this poses a beautiful problem for us to investigate and see if our algorithm will be able to distinguish them.

let's plot the elbow curve to establish if we will get the expected 4 clusters that we have generated.

```
In [ ]: wcss = []
for i in range(1, 11):
    kmeans = KMeans(
        n_clusters=i, init="k-means++", max_iter=300, n_init=10, random_state=42
    )
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.rcParams["figure.figsize"] = (7, 5)
plt.plot(range(1, 11), wcss)
plt.title("K-Means Clustering(The Elbow Method)", fontsize=20)
plt.xlabel("K")
plt.ylabel("WCSS")
plt.suptitle("Fig. 6: The Elbow Method", fontweight="bold", horizontalalignment="right")
plt.show()
```

Note that after the value of $k = 4$, the WCSS decreases very slowly. Therefore, our value of k is equal to 4.

```
In [ ]: kmeans = KMeans(n_clusters=4, init="k-means++", max_iter=300, n_init=10, random_state=42)
y_means = kmeans.fit_predict(X)

plt.rcParams["figure.figsize"] = (10, 10)
plt.title("Cluster of Dataset", fontsize=30)

plt.scatter(X[y_means == 0, 0], X[y_means == 0, 1], s=100, c="pink", label="Cluster 0")
plt.scatter(X[y_means == 1, 0], X[y_means == 1, 1], s=100, c="orange", label="Cluster 1")
plt.scatter(
    X[y_means == 2, 0], X[y_means == 2, 1], s=100, c="lightgreen", label="Cluster 2"
)
plt.scatter(X[y_means == 3, 0], X[y_means == 3, 1], s=100, c="red", label="Cluster 3")
plt.scatter(
    kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=50, c="black"
)

plt.xlabel("First Feature")
plt.ylabel("Second Feature")
plt.legend()
# plt.suptitle('Fig 10: Clustering Results', fontsize=20, y = 1, ha = 'right')
plt.suptitle(
```

```
    "Fig. 7: Clustering Results", fontweight="bold", horizontalalignment="right"
)
plt.show()
```

We have trained the K-means algorithm in our dataset and we can see that the output shows clear distinction between different clusters.

In the next subsection, we will discuss the expectation-maximization approach of finding the composition of the clusters.

3.3 k-Means Algorithm: Expectation–Maximization

The Expectation-Maximization (E-M) procedure is as follows.

1. Randomly select the cluster centers.
2. Repeat the following steps until convergence is reached:
 - E-Step: Points are assigned to the nearest cluster center.
 - M-Step: Find the mean of each cluster and replace the cluster center with the mean.

We can visualize the E-M algorithm in action below.

```
In [ ]: from sklearn.datasets import make_blobs
        from sklearn.metrics import pairwise_distances_argmin

X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=
rng = np.random.RandomState(42)
centers = [0, 4] + rng.randn(4, 2)

def draw_points(ax, c, factor=1):
    ax.scatter(X[:, 0], X[:, 1], c=c, cmap="viridis", s=50 * factor, alpha=0.3)

def draw_centers(ax, centers, factor=1, alpha=1.0):
    ax.scatter(
        centers[:, 0],
        centers[:, 1],
        c=np.arange(4),
        cmap="viridis",
        s=200 * factor,
        alpha=alpha,
    )
    ax.scatter(centers[:, 0], centers[:, 1], c="black", s=50 * factor, alpha=alp

def make_ax(fig, gs):
    ax = fig.add_subplot(gs)
    ax.xaxis.set_major_formatter(plt.NullFormatter())
    ax.yaxis.set_major_formatter(plt.NullFormatter())
    return ax

fig = plt.figure(figsize=(15, 4))
gs = plt.GridSpec(
```

```

    4, 15, left=0.02, right=0.98, bottom=0.05, top=0.95, wspace=0.2, hspace=0.2
)
ax0 = make_ax(fig, gs[:4, :4])
ax0.text(
    0.98,
    0.98,
    "Random Initialization",
    transform=ax0.transAxes,
    ha="right",
    va="top",
    size=16,
)
draw_points(ax0, "gray", factor=2)
draw_centers(ax0, centers, factor=2)

for i in range(3):
    ax1 = make_ax(fig, gs[:2, 4 + 2 * i : 6 + 2 * i])
    ax2 = make_ax(fig, gs[2:, 5 + 2 * i : 7 + 2 * i])

    # E-step
    y_pred = pairwise_distances_argmin(X, centers)
    draw_points(ax1, y_pred)
    draw_centers(ax1, centers)

    # M-step
    new_centers = np.array([X[y_pred == i].mean(0) for i in range(4)])
    draw_points(ax2, y_pred)
    draw_centers(ax2, centers, alpha=0.3)
    draw_centers(ax2, new_centers)
    for i in range(4):
        ax2.annotate(
            "",
            new_centers[i],
            centers[i],
            arrowprops=dict(arrowstyle="->", linewidth=1),
        )

    # Finish iteration
    centers = new_centers
    ax1.text(
        0.95, 0.95, "E-Step", transform=ax1.transAxes, ha="right", va="top", siz
    )
    ax2.text(
        0.95, 0.95, "M-Step", transform=ax2.transAxes, ha="right", va="top", siz
    )

# Final E-step
y_pred = pairwise_distances_argmin(X, centers)
axf = make_ax(fig, gs[:4, -4:])
draw_points(axf, y_pred, factor=2)
draw_centers(axf, centers, factor=2)
axf.text(
    0.98,
    0.98,
    "Final Clustering",
    transform=axf.transAxes,
    ha="right",
    va="top",
    size=16,

```

```

)
plt.suptitle(
    "Fig. 8: Expectation-Maximization", fontweight="bold", horizontalalignment="
)
plt.show()

```

The limitations of the E-M algorithm are:

- We may not achieve a globally optimal solution.
- We must choose the number of clusters before modeling.
- K-means will be slow for big data.
- K-means only works with linear cluster boundaries.

4. Fuzzy K-Means Clustering

Fuzzy clustering is the same as the K-means algorithm with the difference being that each point has some measure (probability) of belonging to each cluster in our dataset. In this case, a point might belong to 2 or more clusters if the point is at the middle of the clusters.

The other processes in K-means such as random initialization, iteration, and algorithm termination are the same in fuzzy K-means. As such, the clusters are treated as probabilistic distributions. By converting the probability function to a binary distribution giving 1 for data points close to a centroid and 0 otherwise, the fuzzy K-means becomes the usual K-means.

Below is the procedure for the fuzzy K-means algorithm;

- First, fix the number of clusters.
- Initialize the centroids randomly and compute the probability of the data points belonging to the different clusters.
- Knowing the probability of each data point belonging to a cluster, we recalculate new centroids by computing the mean of the clusters.
$$v_k(n+1) = \frac{\sum_{x_i \in k} x_i P(v_k|x_i)^b}{\sum_{x_i \in k} P(v_k|x_i)^b}$$
- The algorithm is terminated when it converges or when we reach the number of specified iterations.

5. Hierarchical Clustering

In hierarchical clustering we define a distance matrix using linkage with the following parameters

- Method: To calculate the distance between clusters.
- Metric: The distance metric to be used.
- Optimal ordering: To order data points.

The different types of methods we use in hierarchical clustering are:

1. Single: Finds the distance between the closest points in different clusters.

2. Complete: Finds the distance between the farthest points in different clusters.
3. Average: Finds the distance between the arithmetic mean of our clusters.
4. Centroids: Finds the distance between the geometric mean of our clusters.
5. Median: Finds the distance between the median of our clusters.
6. Ward: Finds distance based on our sum of squares.

Below are Python scripts on how to implement the different methods.

5.1 Ward Method

```
In [ ]: import pandas as pd
        from sklearn.datasets import make_blobs

        X1, y1 = make_blobs(
            n_samples=50, centers=[[4, 4], [-2, -1], [1, 1], [10, 4]], cluster_std=0.9
        )
        df = pd.DataFrame(X1, columns=["X", "y"])
        df.head()
```

```
In [ ]: import seaborn as sns
        from scipy.cluster.hierarchy import fcluster, linkage

        # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="ward", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels"] = fcluster(distance_matrix, 2, criterion="maxclust")

        # Plot clusters
        sns.scatterplot(x="X", y="y", hue="cluster_labels", data=df)
        plt.suptitle("Fig. 9: Ward Method", fontweight="bold", horizontalalignment="right")
        plt.show();
```

5.2 Single Method

```
In [ ]: # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="single", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels_2"] = fcluster(distance_matrix, 2, criterion="maxclust")

        # Plot clusters
        sns.scatterplot(x="X", y="y", hue="cluster_labels_2", data=df)
        plt.suptitle("Fig. 10: Single Method", fontweight="bold", horizontalalignment="right")
        plt.show();
```

5.3 Complete Method

```
In [ ]: # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="complete", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels_3"] = fcluster(distance_matrix, 2, criterion="maxclust")
```

```
# Plot clusters
sns.scatterplot(x="X", y="y", hue="cluster_labels", data=df)
plt.suptitle("Fig. 11: Complete Method", fontweight="bold", horizontalalignment="right")
plt.show();
```

5.4 Dendrogram

Dendrograms help us visualize how clusters are formed, and from them, we can choose the number of clusters in our dataset. Below is simple code that gives us a dendrogram diagram.

```
In [ ]: from scipy.cluster.hierarchy import dendrogram

# Create a dendrogram
dn = dendrogram(distance_matrix)
plt.suptitle("Fig. 11: Dendrogram", fontweight="bold", horizontalalignment="right")
plt.show()
```

In summary, the following are steps taken by hierarchical clustering:

1. We treat each point in our dataset as its own cluster.
2. We then calculate the Euclidean distance between the centroids of all clusters in our dataset.
3. We join the closest clusters together.
4. We repeat steps 2 and 3 until we get only one cluster that contains all the data points.
5. A dendrogram that shows the evolution of the hierarchical structure is plotted showing how clusters are arranged from top to bottom.
6. It is now up to us to decide at which level we want to create our clusters.

6. Gaussian Mixture Models (GMM)

This is a model-based approach that uses several models to generate clusters and attempts to optimize the model fit on the data.

The advantages of this type of clustering includes that the possibility of one point belonging to several clusters, a density approximation for each cluster, the availability of robust statistical inference methods to do our clustering, and the freedom to choose the component distribution of the model.

The K component distributions that make up a mixture model are collectively combined to give $f(x) = \sum_{k=1}^K a_k f_k(x)$ where a_k is the k -th component contribution in building $f(x)$.

Like the K-means algorithm above, GMM is also used to find the number of clusters in a dataset. Consider the example below.

```
In [ ]: %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
```

```
import seaborn as sns

# Generate some data
from sklearn.datasets import make_blobs

sns.set()

X, y_true = make_blobs(n_samples=400, centers=4, cluster_std=0.60, random_state=
X = X[:, ::-1] # flip axes for better plotting
```

To visualize the number of clusters, we fit our Gaussian mixture model with a pre-specified number of clusters.

```
In [ ]: from sklearn import mixture

gmm = mixture.GaussianMixture(n_components=4).fit(X)
labels = gmm.predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap="viridis")
plt.suptitle(
    "Fig. 12: Clusters from GMM", fontweight="bold", horizontalalignment="right"
)
plt.show();
```

The Gaussian mixture model contains a probabilistic model that gives the probability of a point belonging to a cluster.

```
In [ ]: probs = gmm.predict_proba(X)
print(probs[:5].round(3))
```

The Gaussian mixture model like the k-means algorithm uses the expectation-maximization approach below.

1. We choose a random starting point.
2. Repeat the following until convergence.
 - E-Step: Find the weight of each point belonging to a given cluster.
 - M-Step: Update the location and shape of each cluster based on the data point weights.

GMM will at times miss the globally optimal solution, and it is therefore recommended to use different random initialization.

6.1 Number of Clusters

GMM being a generative model will provide us with a natural technique of finding the optimal number of clusters. Since a generative model provides probability distribution for a given dataset, we can use analytic criterion techniques like *Akaike Information Criterion (AIC)* or *Bayesian Information Criterion (BIC)* to reduce the chances of our model overfitting. We can also use cross-validation to achieve the same.

In the example below, we use AIC and BIC.

```
In [ ]: from sklearn.datasets import make_moons

Xmoon, ymoon = make_moons(200, noise=0.05, random_state=0)
plt.scatter(Xmoon[:, 0], Xmoon[:, 1])

n_components = np.arange(1, 21)
models = [
    mixture.GaussianMixture(n, covariance_type="full", random_state=0).fit(Xmoon)
    for n in n_components
]

plt.plot(n_components, [m.bic(Xmoon) for m in models], label="BIC")
plt.plot(n_components, [m.aic(Xmoon) for m in models], label="AIC")
plt.legend(loc="best")
plt.xlabel("n_components")
plt.suptitle(
    "Fig. 13: Optimal Clusters", fontweight="bold", horizontalalignment="right"
);
```

7. DBSCAN

DBSCAN is short for Density-Based Spatial Clustering of Applications with Noise. It works by identifying regions that are densely populated from sparsely populated regions. The logic follows that points belonging to a cluster should be close to the points in the same cluster.

It is defined by two parameters:

1. **Epsilon** that finds the radius, which has enough points within.
2. **Minimum Samples** that finds the minimum number of points we want to be in a cluster.

Below is an implementation of the DBSCAN algorithm.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

%matplotlib inline
```

```
In [ ]: def createDataPoints(centroidLocation, numSamples, clusterDeviation):
    # Create random data and store in feature matrix X and response vector y.
    X, y = make_blobs(
        n_samples=numSamples, centers=centroidLocation, cluster_std=clusterDeviation
    )

    # Standardize features by removing the mean and scaling to unit variance
    X = StandardScaler().fit_transform(X)
    return X, y
```

```
In [ ]: X, y = createDataPoints([[4, 3], [2, -1], [-1, 4]], 1500, 0.5)
```

```

In [ ]: epsilon = 0.3
        minimumSamples = 7
        db = DBSCAN(eps=epsilon, min_samples=minimumSamples).fit(X)
        labels = db.labels_
        labels

In [ ]: # First, create an array of booleans using the labels from db.
        core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
        core_samples_mask[db.core_sample_indices_] = True
        core_samples_mask

In [ ]: # Number of clusters in labels, ignoring noise if present.
        n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
        n_clusters_

In [ ]: # Remove repetition in labels by turning it into a set.
        unique_labels = set(labels)
        unique_labels

In [ ]: # Create colors for the clusters.
        colors = plt.cm.Spectral(np.linspace(0, 1, len(unique_labels)))
        colors

In [ ]: # Plot the points with colors
        for k, col in zip(unique_labels, colors):
            if k == -1:
                # Black used for noise.
                col = "k"

            class_member_mask = labels == k

            # Plot the datapoints that are clustered
            xy = X[class_member_mask & core_samples_mask]
            plt.scatter(xy[:, 0], xy[:, 1], s=50, c=col, marker="o", alpha=0.5)

            # Plot the outliers
            xy = X[class_member_mask & ~core_samples_mask]
            plt.scatter(xy[:, 0], xy[:, 1], s=50, c=col, marker="o", alpha=0.5)
        plt.show()

```

8. Conclusion

We have introduced the concept of clustering and have given an overview of the clustering techniques applied to a dataset. We also illustrated a simple application of K-means clustering to datasets. In the next lesson, we are going to apply hierarchical clustering in a real world dataset.

References

1. Chang, Chih-Tang, Jim ZC Lai, and Mu-Der Jeng. "A fuzzy k-means clustering algorithm using cluster center displacement." *J. Inf. Sci. Eng.* 27.3 (2011): 995-1009.
2. Cohn, Ryan, and Elizabeth Holm. "Unsupervised Machine Learning via Transfer Learning and k-means Clustering to Classify Materials Image Data." *Integrating*

- Materials and Manufacturing Innovation*, vol. 10, no. 2, 2021, pp. 231-244.
3. Davis, Damek, Mateo Diaz, and Kaizheng Wang. "Clustering a mixture of gaussians with unknown covariance." arXiv preprint arXiv:2110.01602 (2021).
 4. Ghahramani, Zoubin. "Unsupervised Learning." *Advanced Lectures on Machine Learning*, edited by Oliver Bousquet, Ulrike von Luxburg, and Gunnar Rätsch, Springer, 2003, pp. 72-112.
 5. Goldberger, Jacob, and Sam Roweis. "Hierarchical Clustering of a Mixture Model." *Advances in Neural Information Processing Systems*, vol. 17, 2004.
 6. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, 2012, pp. 86-97.
 7. Nielsen, Frank. *Introduction to HPC with MPI for Data Science*. Springer, 2016.
-